

Day 2

duplicateCheck.java
find second largest Element

Big O

drop constant factors $\rightarrow 5n$

$O(n)$, $n^2/n \rightarrow O(n^2)$

Sequential block add
a loop of n
then, loop it m

$O(n \times m)$

nested loop multiply
a loop of n inside a loop of n

~~$O(n^2)$~~

different inputs different letters
two separate array of sizes n and m
give $O(n \times m)$ not $O(n^2)$

// **Pattern 1** single loop $O(n)$

```
for (int i = 0; i < n; i++)
```

```
count++
```

// **Pattern 2** nested independent $O(n^2)$

```
for (int i = 0; i < n; i++)
```

```
for (int j = 0; j < n; j++)
```

```
count++
```

// **Pattern 3** nested, inner depend on outer

$$O(n)^2$$

```
for(int i=0; i<n; i++)
    for(int j=0; j<i; j++)
        count++
```

Pattern 4 halving ~~$O(\log n)$~~ $O(\log n)$

```
for(int i=n; i>1; i/=2)
```

 $O(\log n)$

Pattern 5 Linear outer $\times$ logarithmic inner $O(n \log n)$

```
for(int i=0; i<n; i++)
    for(int j=1; j<n; j*=2)
```

 $O(n \log n)$

Pattern 6 Two loop one after other $O(n)$

```
for(int i=0; i<n; i++)
    count++;
for(int i=0; i<n; i++)
    count++
```

 $O(n)$

Pattern 7 loop until $i*i$ $O(\sqrt{n})$

```
for(int i=1; i*i<=n; i++)
    count++
```

Pattern 8 fixed number of iterations $O(1)$

```
for(int i=0; i<100; i++)
```

1) $(int\ i = 1; i < n; i *= 3)$ $O(\log n)$
 i multiplies by 3 each time so it
 run $\log_3 n$ times.

2) $for(int\ i = 0; i < n; i++)$
 $for(int\ j = 0; j < i + 5; j++)$ $O(n)$
 because inner loop exactly run 5 times

3)

1) LoopComplexity.java

2) System.nanoTime use GrowthExpenin

3) duplicateCheck.java
 StringConcat.java

Day 2

Yesterday
I ~~was~~ we learn
Stack

Page No.	
Date	

Implement

- ✓ 1) Implement stack through Array.
- 2) Complexity analysis of loop and recursive algorithm
- 3) implement queues inserting element at different location first Last)
- 4) Implement circular queue.

```
import java.util.Scanner;  
public class Stack ArrayIntDemo {
```

```
    private final int[] stack;  
    private int top = -1;
```

```
    StackArrayIntDemo(int capacity) {  
        stack = new int[capacity];
```

```
    }
```

```
    void push(int value) {
```

```
        if (top == stack.length - 1) {
```

```
            System.out.println("Stack  
OverFlow");
```

```
            return;
```

```
        }
```

```
        stack[++top] = value;
```

```
        System.out.println(value +  
            "pushed onto stack");
```

```
    }
```

```
}
```

```

int pop() {
    if (isEmpty()) {
        System.out.println(
            "Stack overflow");
        return -1;
    }
    return stack[top--];
}

```

```

int peek() {
    if (isEmpty()) {
        return -1;
    }
    return stack[top];
}

```

```

boolean isEmpty() {
    return top == -1;
}

```

```

boolean isFull() {
    return top == stack.
        length - 1;
}

```

```

void display() {
    if (isEmpty()) {
        SOP("Stack is empty");
        return;
    }
}

```

```
System.out.print("Stack Element  
(top to bottom):");
```

```
for (int i = top; i >= 0; i--) {
```

```
    System.out.print(Stack[i]  
        + " ");
```

```
}
```

```
System.out.println();
```

```
}
```

```
public static void main(String[] args) {
```

```
    Scanner sc = new Scanner  
        (System.in);
```

```
    System.out.println("Enter capacity");
```

```
    int capacity = sc.nextInt();
```

```
    if (capacity <= 0) {
```

```
        System.out.println("Capacity  
        must be greater than  
        zero");
```

```
        sc.close();
```

```
        return;
```

```
}
```

```
StackArrayIntroDemo Stack =
    new StackArrayIntroDemo(capacity);
```

```
while (true) {
    System.out.println();
    System.out.println("Integer
                        Stack using Array");
```

```
    System.out.println("1. push");
    System.out.println("2. pop");
    System.out.println("3. peek");
    System.out.println("4. display");
    System.out.println("5. exit");
```

```
    System.out.println("Enter your
                        choice");
```

```
int choice;
    int choice = Sc.nextInt();
```

```
    Switch(choice) {
        case 2:
```

```
        System.out.println("Enter
                            value to pop");
```

```
        Stack.push(Scanner Sc.next());
        break;
```

C8

case 2 :

```
if (!stack.isEmpty()) {
```

```
    System.out.println (
```

```
        stack.pop() + "
```

```
        popped from stack");
```

```
    }
```

```
else
```

```
    stack.pop();
```

```
}
```

```
break;
```

case 3 :

```
if (stack.isEmpty()) {
```

```
    System.out.println
```

```
        ("Stack is empty");
```

```
else {
```

```
    System.out.println ("Top
```

```
    element: " + stack.
```

```
    peek());
```

```
}
```

```
break;
```

case 4 :

```
stack.display();
```

```
break;
```